LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK

“TRANSAKSI DINAMIS (POLYMORPHISM)”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Nurfiah, S.ST, M.Kom

Asisten Praktikum:

Siti Fadhilah Rahmi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. Langkah Kerja Praktikum

a. Langkah pertama yaitu, buka aplikasi yang dibutuhkan untuk praktikum (Eclipse) Lalu buat package pekan 5 dan berisi class Rekening, Main, Transaksi yang menggunakan kode program pada pekan sebelumnya dan juga terdapat class baru yang diberikan pada modul ini.

b. Langkah kedua, membuat kode program yang terdapat penambahan atau perubahan di dalam modul praktikum :

1. Class Rekening :

```
1.  // Mengubah void menjadi boolean
2.      public boolean tarikTunai(double nominal) {
3.              if (nominal <= 0) {
4.                      System.out.println("Gagal: Nominal tarik harus
    lebih dari 0!");
5.                      return false;
6.              } else if (nominal > saldo){
7.                      System.out.println("Transaksi Gagal: Saldo tidak
    mencukupi. Saldo Anda: Rp" + saldo);
8.                      return false;
9.              } else {
10.                     saldo -= nominal;
11.
12.                     String idTrx = "TRX-T-" +
    System.currentTimeMillis();
13.                     riwayatTransaksi.add(new Transaksi(idTrx,
    "Debit", nominal));
14.
15.                     System.out.println("Tarik tunai Rp" + nominal +
    " berhasil. Sisa saldo: Rp" + saldo);
16.                     return true; // Transaksi sukses
17.             }
18.     }
```

Kode Pekan_5 ini adalah method tarikTunai() pada class Rekening yang tipe kembaliannya diubah dari void (Pekan_4) menjadi boolean, sehingga method melaporkan hasil transaksi ke pemanggilnya: true jika berhasil dan false jika gagal. Dengan begitu, Main atau subclass dapat mengambil keputusan berdasarkan hasil penarikan, tidak hanya mengandalkan pesan di layar.

Validasi diubah dari satu kondisi nominal >= 10000 && nominal <= getSaldo() dengan satu pesan kegagalan, menjadi struktur if-else if-else yang lebih spesifik. Kondisi pertama menolak nominal kurang dari atau sama dengan 0, dan kondisi kedua menolak nominal yang melebihi saldo, masing-masing dengan pesan sesuai penyebabnya lalu mengembalikan false. Batas minimal Rp10.000 juga dihapus agar method lebih umum, sehingga aturan khusus seperti limit overdraft pada RekeningGiro dapat ditentukan oleh subclass melalui override.

Jika penarikan valid, saldo dikurangi langsung dengan saldo -= nominal, bukan lagi lewat setSaldo(getSaldo() - nominal), karena atribut saldo bersifat protected sehingga lebih ringkas. Transaksi tetap dicatat dengan membuat id unik berawalan "TRX-T-" dari System.currentTimeMillis(), lalu objek Transaksi berjenis "Debit" langsung ditambahkan ke riwayatTransaksi tanpa variabel perantara. Terakhir, program menampilkan pesan sukses dengan redaksi "Sisa saldo" (sebelumnya "Saldo saat ini") dan mengembalikan true.

2. Class Main

Untuk kelas Main tidak ada perubahan dari modul, tetapi terdapat perubahan pada bagian tugas/latihan praktikum. Jadi kelas main masih sama seperti pada pekan sebelumnya.

3. Class Transaksi

Untuk kelas Transaksi tidak ada perubahan dari modul, Jadi kelas Transaksi masih sama seperti pada pekan sebelumnya.

4. Class RekeningTabungan

Untuk kelas RekeningTabungan tidak ada perubahan dari modul, Jadi kelas RekeningTabungan masih sama seperti pada pekan sebelumnya

5. Class RekeningGiro

```
1. package Pekan_5;
2.
3. public class RekeningGiro extends Rekening{
```

```
4.     private double batasOverdraft;
5.
6.     public RekeningGiro (String nomor, String nama, double
   saldoAwal, String pinAwal, double batasOverdraft) {
7.           // Memanggil inisialisasi dasar dari Superclass
8.           super(nomor, nama, saldoAwal, pinAwal);
9.           this.batasOverdraft = batasOverdraft;
10.    }
11.
12.    // IMPLEMENTASI POLIMORFISME (OVERRIDING)
13.    @Override
14.    public boolean tarikTunai(double nominal) {
15.          if (nominal <= 0) {
16.                System.out.println("Gagal: Nominal tarik Giro
   harus lebih dari 0!");
17.                return false;
18.          }
19.          // Logika Khusus Giro: Boleh ditarik asalkan nominal
   tidak melebihi (saldo + Overdraft)
20.          else if (nominal > (saldo + batasOverdraft)) {
21.                System.out.println("Transaksi Gagal: Melebihi
   batas overdraft! Maksimal penarikan: Rp" + (saldo +
   batasOverdraft));
22.                return false;
23.          } else {
24.                saldo -= nominal; // Saldo bisa bernilai negatif
   disini (memakan overdraft)
25.
26.                String idTrx = "TRX-G-" +
   System.currentTimeMillis();
27.                riwayatTransaksi.add(new Transaksi(idTrx, "Debit
   Giro", nominal));
28.
29.                System.out.println("Tarik tunai Giro Rp" +
   nominal + " berhasil. Saldo saat ini: Rp" + saldo);
30.                return true;
31.          }
32.    }
33.
34.    public double getBatasOverdraft() {
35.          return batasOverdraft;
36.    }
37.    // Catatan Penarikan hingga limit overdraft akan kita
   selesaikan di modul 5
38.
39. }
```

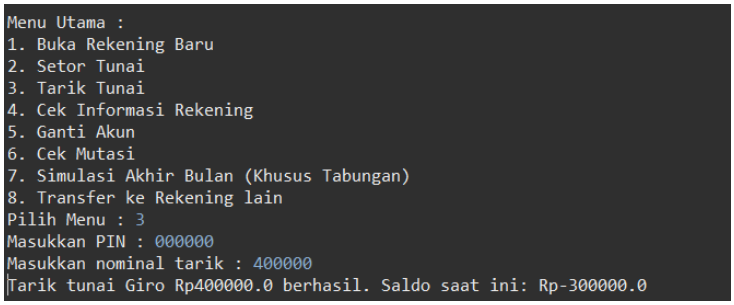
Kode Pekan_5 ini adalah class RekeningGiro, subclass dari Rekening (extends), yang dibanding Pekan_4 mendapat tambahan method tarikTunai() dengan anotasi @Override. Pada Pekan_4, class ini hanya menyimpan atribut private batasOverdraft (batas maksimal saldo boleh minus) beserta getter-nya,

sehingga penarikan masih memakai aturan bawaan Rekening yang tidak mengizinkan saldo negatif. Constructor-nya memanggil super(nomor, nama, saldoAwal, pinAwal) untuk menginisialisasi data dasar dari Rekening, lalu mengisi batasOverdraft yang khusus dimiliki Giro.

Method tarikTunai() menerapkan polimorfisme (overriding): nama dan tipe kembaliannya (boolean) sama dengan milik Rekening, tetapi perilakunya disesuaikan dengan rekening giro. Nominal kurang dari atau sama dengan 0 ditolak dengan pesan khusus Giro dan mengembalikan false. Perbedaan terpentingnya ada pada kondisi nominal > (saldo + batasOverdraft), yaitu penarikan hanya ditolak jika melebihi saldo ditambah batas overdraft (bukan hanya saldo), dengan pesan yang menampilkan jumlah maksimal yang boleh ditarik, lalu mengembalikan false. Jika valid, saldo dikurangi langsung dengan saldo -= nominal (bisa karena saldo bersifat protected) dan boleh bernilai negatif karena memakai fasilitas overdraft.

Transaksi dicatat dengan id berawalan “TRX-G-” dan jenis “Debit Giro”, berbeda dari rekening biasa (“TRX-T-” dan “Debit”), sehingga mutasi dapat membedakannya. Setelah itu ditampilkan pesan sukses beserta saldo saat ini dan method mengembalikan true. Terakhir, getter getBatasOverdraft() tetap dipertahankan, sedangkan komentar “akan diselesaikan di modul 5” sudah tidak relevan dan sebaiknya dihapus karena fiturnya kini sudah diimplementasikan.

6. Uji coba



Gambar ini menunjukkan hasil eksekusi program Pekan_5 saat pengguna memilih menu 3 (Tarik Tunai) pada akun bertipe RekeningGiro.

Setelah PIN (000000) diterima oleh method otentikasi(), pengguna menarik Rp400.000 dan program menampilkan pesan “Tarik tunai Giro Rp400000.0 berhasil”. Kata “Giro” pada pesan menandakan bahwa yang dijalankan adalah method tarikTunai() versi override milik RekeningGiro, bukan milik Rekening biasa.

Hal terpenting dari output ini adalah saldo akhir yang bernilai negatif, yaitu Rp-300000.0. Pada rekening biasa, penarikan melebihi saldo akan ditolak, tetapi pada rekening giro penarikan hanya ditolak jika nominal melebihi saldo + batasOverdraft. Karena saldo menjadi -300.000 setelah penarikan 400.000, saldo awalnya adalah Rp100.000 dan batas overdraft minimal Rp300.000, sehingga penarikan masih diizinkan dengan memakai fasilitas overdraft.

2. Tugas/Latihan Praktikum

1. Buat Method Transfer : Di dalam class Rekening.java, tambahkan satu method bar dengan nama : public void transfer(Rekening tujuan, double nominal, String pinInput)
   a. Method ini menerima parameter polimorfik Rekening tujuan, bisa menerima Tabungan atau Giro.
   b. Logika Method:
      - Pertama, lakukan otentikasi(pinInput). Jika false, batalkan.
      - Kedua, panggil this.tarikTunai(nominal). (overriding, jika this adalah Giro, ia otomatis menggunakan aturan overdraft).
      - Ketiga, periksa kembalian nilai tarikTunai. Jika true, panggil tujuan.setorTunai(nominal).
2. Integrasi ke CLI Utama: Buka Main.java dan tambahkan menu 8. Transfer ke Rekening Lain.
   a. Minta pengguna memasukkan Nomor Rekening Tujuan.
   b. Cari nomor tersebut di dalam daftarRekening.

c. Jika ketemu, minta nominal transfer dan PIN, lalu panggil akunAktif.transfer(akunTujuan, nominal, pin).

Jawaban :

1. Kode program class Rekening :

```
1.  public void transfer(Rekening tujuan, double nominal, String
    pinInput) {
2.          if (!otentikasi(pinInput)) {
3.              System.out.println("Pin salah!");
4.              return;
5.          }
6.          if (this.tarikTunai(nominal)) {
7.              tujuan.setorTunai(nominal);
8.          }
9.      }
```

Kode ini adalah method transfer() pada class Rekening Pekan_5, yang memindahkan dana dari rekening pengirim (objek yang memanggil method) ke rekening tujuan. Method bertipe void dan menerima tiga parameter: tujuan (objek Rekening penerima), nominal (jumlah transfer), dan pinInput (PIN pengirim untuk verifikasi).

Baris if (!otentikasi(pinInput)) memeriksa PIN pengirim; jika salah, program menampilkan “Pin salah!” lalu return menghentikan method sehingga tidak ada dana yang berpindah. Jika PIN benar, baris if (this.tarikTunai(nominal)) menarik dana dari rekening pengirim sekaligus memanfaatkan nilai boolean yang dikembalikan: transfer hanya dilanjutkan bila penarikan berhasil (true), sedangkan bila gagal (nominal tidak valid atau saldo tidak cukup) tidak ada setoran ke tujuan, sehingga uang tidak muncul begitu saja. Terakhir, tujuan.setorTunai(nominal) menambah saldo penerima dan mencatatnya sebagai transaksi “Kredit” di riwayatnya.

Method ini juga menerapkan polimorfisme, karena this.tarikTunai() menjalankan versi sesuai jenis objek pengirim: pada RekeningGiro transfer boleh memakai fasilitas overdraft, sedangkan pada rekening biasa dibatasi oleh saldo. Satu catatan, sisi pengirim tercatat sebagai “Debit” (atau “Debit

Giro"), bukan "Transfer", sehingga di mutasi tidak terlihat bahwa dana tersebut berasal dari proses transfer.

2. Kode program class Main :

```
1. System.out.println("8. Transfer ke Rekening lain");
2. 
3. case 8:
4.     if (akunAktif == null) {
5.             System.out.println("Error : Anda belum memiliki akun
   aktif!");
6.     } else {
7.             System.out.print("Masukkan Nomor Rekening Tujuan : ");
8.             String nomorTujuan = input.nextLine();
9. 
10.            Rekening akunTujuan = null;
11.            for (Rekening rA : rekeningAktif) {
12.                   if (rA.cocokDenganNomor(nomorTujuan)) {
13.                          akunTujuan = rA;
14.                          break;
15.                   }
16.            }
17. 
18.            if (akunTujuan == null) {
19.                   System.out.println("Gagal: Rekening tujuan tidak
   ditemukan.");
20.            } else if (akunTujuan == akunAktif) {
21.                   System.out.println("Gagal: Tidak dapat transfer
   ke rekening sendiri.");
22.            } else {
23.                   System.out.print("Masukkan Nominal Transfer :
   ");
24.                   double nominal = input.nextDouble();
25.                   input.nextLine();
26. 
27.                   System.out.print("Masukkan PIN : ");
28.                   String pin1 = input.nextLine();
29. 
30.                   akunAktif.transfer(akunTujuan, nominal, pin1);
31.    }
32. }
33. break;
```

Kode ini adalah potongan menu 8 (Transfer ke Rekening lain) pada Main Pekan_5. Baris System.out.println("8. Transfer ke Rekening lain") menambahkan pilihan baru pada menu utama, sedangkan case 8 menjalankan proses transfer melalui method transfer() milik class Rekening. Program pertama-tama mengecek akunAktif; jika masih null, ditampilkan pesan error karena transfer membutuhkan akun pengirim yang aktif.

Jika akun aktif ada, program meminta nomor rekening tujuan dan menyimpannya di nomorTujuan. Variabel akunTujuan disiapkan bernilai null sebagai penampung hasil pencarian, lalu list rekeningAktif ditelusuri dengan for-each. Setiap rekening diperiksa dengan cocokDenganNomor(nomorTujuan), dan jika cocok, objeknya disimpan ke akunTujuan lalu break menghentikan pencarian.

Hasil pencarian divalidasi dengan struktur if-else if-else. Jika akunTujuan == null, ditampilkan pesan bahwa rekening tujuan tidak ditemukan. Jika akunTujuan == akunAktif, ditampilkan pesan bahwa transfer ke rekening sendiri tidak diperbolehkan; pengecekan ini membandingkan referensi objek sehingga dana tidak ditarik lalu disetor kembali ke rekening yang sama. Jika keduanya tidak terpenuhi, program meminta nominal dengan nextDouble(), diikuti input.nextLine() untuk membersihkan sisa enter di buffer agar input PIN tidak terlewat. Program lalu meminta PIN pengirim (disimpan di pin1) dan memanggil akunAktif.transfer(akunTujuan, nominal, pin1), yang memverifikasi PIN, menarik dana dari pengirim, lalu menyetorkannya ke rekening tujuan. Terakhir, break mengakhiri case 8 agar program kembali ke menu utama.

Output :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening lain
Pilih Menu : 8
Masukkan Nomor Rekening Tujuan : 001
Masukkan Nominal Transfer : 50000
Masukkan PIN : 000000
Tarik tunai Rp50000.0 berhasil. Sisa saldo: Rp150000.0
Setor tunai Rp50000.0 berhasil. Saldo saat ini : Rp70000.0
```

Saldo awal akun yang menjadi pengirim : Rp200.000 menjadi Rp150.000

Saldo awal akun yang menjadi tujuan: Rp20.000 menjadi Rp70.000

Jika mengirim pada nomor rekening yang akun nya sedang digunakan :

```
Menu Utama :
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cek Mutasi
7. Simulasi Akhir Bulan (Khusus Tabungan)
8. Transfer ke Rekening lain
Pilih Menu : 8
Masukkan Nomor Rekening Tujuan : 000
Gagal: Tidak dapat transfer ke rekening sendiri.
```